EN3160 — Assignment 02: Fitting and Alignment

Index number: 230070T
Name: Balasooriya BRBD **GitHub profile:** https://github.com/banula33/A02

Question 1 — Sunflower detection

Use scale-normalized LoG, $\sigma^2\nabla^2(G_\sigma*I)$, to find dark flower centres. Maxima are checked across position and scale. The radius is approximately $\sqrt{2}\sigma$. We use 25 scales from 1.2 to 18 pixels and compare detections before and after checking for surrounding yellow petals. Circles describe the dark centres, not the full petal boundary.

In [2]:
```
%matplotlib inline

import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

# Read at quarter resolution, as given in the assignment.
im = cv.imread('images/the_berry_farms_sunflower_field.jpeg', cv.IMREAD_REDU
gray = cv.cvtColor(im, cv.COLOR_BGR2GRAY).astype(float) / 255
sigmas = np.geomspace(1.2, 18, 25)

# Apply Gaussian smoothing followed by the Laplacian at each scale.
responses = []
for sigma in sigmas:
    blurred = cv.GaussianBlur(gray, (0, 0), sigma)
    log_image = sigma**2 * cv.Laplacian(blurred, cv.CV_64F, ksize=1)
    responses.append(log_image)
responses = np.array(responses)

# Yellow pixels help distinguish flowers from dark leaves.
colour = im.astype(float) / 255
blue, green, red = cv.split(colour)
yellow = (red > 1.8 * blue) & (green > 1.8 * blue)
yellow = yellow & (red > 0.85 * green) & (red > 0.30) & (green > 0.25)

# Find the largest response in each 3 x 3 spatial neighbourhood.
local_max = []
for response in responses:
    local_max.append(cv.dilate(response, np.ones((3, 3), np.uint8)))

candidates = []
threshold = 0.25 * responses.max()
for k in range(1, len(sigmas) - 1):
    # Compare with neighbouring pixels at the current and adjacent scales.
    peaks = (responses[k] == local_max[k])
    peaks = peaks & (responses[k] >= local_max[k - 1])
    peaks = peaks & (responses[k] >= local_max[k + 1])
```

```
    peaks = peaks & (responses[k] > threshold)

    for y, x in np.argwhere(peaks):
        radius = np.sqrt(2) * sigmas[k]
        size = max(2, int(1.8 * radius))
        height, width = gray.shape
        if min(x, y, width - 1 - x, height - 1 - y) < size:
            continue

        # Check a ring around the centre for yellow petals.
        yy, xx = np.ogrid[-size:size+1, -size:size+1]
        distance = xx**2 + yy**2
        ring = (distance >= (0.9 * radius)**2) & (distance <= size**2)
        patch = yellow[y-size:y+size+1, x-size:x+size+1]
        yellow_fraction = patch[ring].mean()
        candidates.append([x, y, radius, sigmas[k], responses[k, y, x], yell

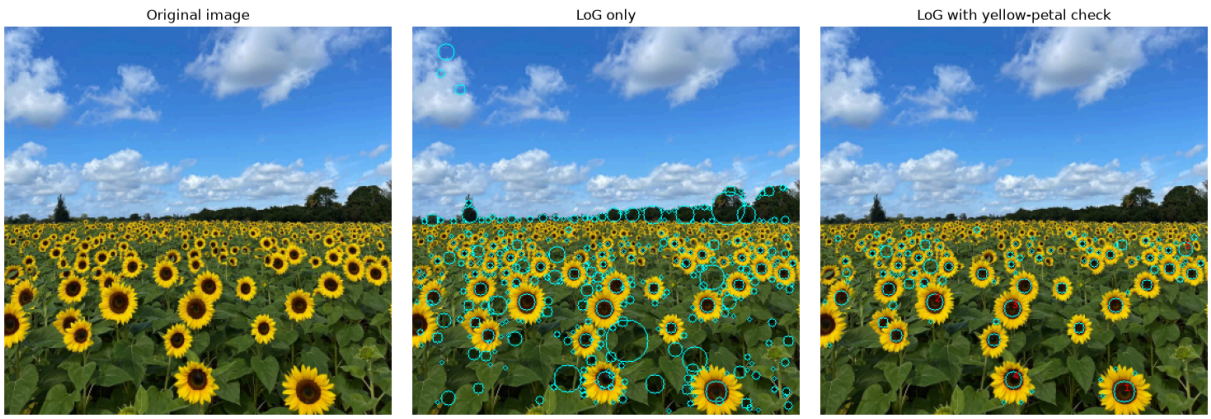
# Keep stronger circles first and remove nearby duplicates.
def remove_duplicates(circles):
    circles = sorted(circles, key=lambda circle: circle[4], reverse=True)
    kept = []
    for circle in circles:
        duplicate = False
        for other in kept:
            distance = np.hypot(circle[0] - other[0], circle[1] - other[1])
            if distance <= 0.85 * max(circle[2], other[2]):
                duplicate = True
                break
        if not duplicate:
            kept.append(circle)
    return kept

baseline = remove_duplicates(candidates)
yellow_candidates = []
for circle in candidates:
    if circle[5] >= 0.60:
        yellow_candidates.append(circle)
flowers = remove_duplicates(yellow_candidates)
largest = sorted(flowers, key=lambda circle: circle[2], reverse=True)[:5]

# Draw the detections using OpenCV circles.
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(cv.cvtColor(im, cv.COLOR_BGR2RGB))
axes[0].set_title('Original image')
for ax, circles, title in [(axes[1], baseline, 'LoG only'),
                           (axes[2], flowers, 'LoG with yellow-petal check')
    result = im.copy()
    for x, y, radius, sigma, response, fraction in circles:
        cv.circle(result, (int(x), int(y)), round(radius), (255, 255, 0), 1)
    if circles is flowers:
        for i, circle in enumerate(largest, 1):
            cv.putText(result, str(i), (int(circle[0]), int(circle[1])),
                       cv.FONT_HERSHEY_SIMPLEX, 0.35, (0, 0, 255), 1)
    ax.imshow(cv.cvtColor(result, cv.COLOR_BGR2RGB))
    ax.set_title(title)
for ax in axes:
```

```
    ax.axis('off')
plt.tight_layout()
plt.show()

print('Sigma range:', sigmas[0], 'to', sigmas[-1], 'pixels')
print('Number of scales:', len(sigmas))
print('LoG detections:', len(baseline))
print('Detections after yellow-petal check:', len(flowers))
print('Largest detected circles (quarter-resolution coordinates):')
for i, circle in enumerate(largest, 1):
    x, y, radius, sigma = circle[:4]
    print(i, ': centre =', (int(x), int(y)), ', radius =', round(radius, 2),
          ', sigma =', round(sigma, 2))
```




```
Sigma range: 1.2 to 18.0 pixels
Number of scales: 25
LoG detections: 356
Detections after yellow-petal check: 110
Largest detected circles (quarter-resolution coordinates):
1 : centre = (282, 338) , radius = 10.32 , sigma = 7.3
2 : centre = (106, 255) , radius = 8.24 , sigma = 5.82
3 : centre = (178, 261) , radius = 8.24 , sigma = 5.82
4 : centre = (179, 327) , radius = 8.24 , sigma = 5.82
5 : centre = (339, 206) , radius = 7.36 , sigma = 5.2
```

Discussion: The yellow-petal check removes many responses from leaves and background texture. Large foreground centres are easier to detect than small distant ones. Shadows, overlapping flowers and tilted heads can still cause errors. The printed radii are in the reduced image; multiply them by four for approximate original-image radii. A lower response threshold gives more detections but also more false positives.

Question 2 — Line and circle fitting with RANSAC

For the line, use $ax+by=d$ with $a^2+b^2=1$ and perpendicular error $|ax+by-d|$. For the circle, use radial error $|\|p-c\|-r|$. Thresholds are 1.5 and 1.3 respectively, about twice the corresponding noise standard deviations. Minimum consensuses are 35 line points and 25 circle points, allowing some circle points to be removed near intersections. Use 2,000 random trials, then refit the winning inliers once using SVD for the line and `minimize` for the circle.

Run parts (a), (b) and (c) in order because each uses results from the previous part.

(a) Estimate the line

Generate the noisy points, find the line consensus using RANSAC, and refit it with total least squares. The normal is kept at unit length.

In [3]:
```
%matplotlib inline

import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize

# Generate the same noisy line and circle described in Listing 1.
np.random.seed(0)
N = 100
half_n = N // 2
r = 10
x0_gt, y0_gt = 2, 3
s = r / 16
t = np.random.uniform(0, 2 * np.pi, half_n)
n = s * np.random.randn(half_n)
x = x0_gt + (r + n) * np.cos(t)
y = y0_gt + (r + n) * np.sin(t)
X_circ = np.column_stack((x, y))
s = 1.0
m, b = -1, 2
x = np.linspace(-12, 12, half_n)
y = m * x + b + s * np.random.randn(half_n)
X_line = np.column_stack((x, y))
X = np.vstack((X_circ, X_line))

# (a) Line RANSAC: choose two points and count nearby points.
rng = np.random.default_rng(101)
best_count = 0
best_error = np.inf
for trial in range(2000):
    sample = rng.choice(len(X), 2, replace=False)
    p1, p2 = X[sample]
    dx, dy = p2 - p1
    normal = np.array([-dy, dx])
    length = np.linalg.norm(normal)
    if length < 1e-10:
        continue
    normal = normal / length  # Required unit normal.
    d = np.dot(normal, p1)
    errors = np.abs(X @ normal - d)
    inliers = errors <= 1.5
    count = inliers.sum()
    error = np.sum(errors[inliers]**2)
    if count >= 35 and (count > best_count or (count == best_count and error
        best_count = count
        best_error = error
        line_mask = inliers
```

```
        line_sample = sample
        sample_line = np.array([normal[0], normal[1], d])
if best_count == 0:
    raise RuntimeError('No line with enough inliers was found.')

# Refit the line using all its inliers (total least squares).
line_points = X[line_mask]
mean = line_points.mean(axis=0)
U, S, Vt = np.linalg.svd(line_points - mean, full_matrices=False)
a, b = Vt[-1]
d = a * mean[0] + b * mean[1]
if d < 0:
    a, b, d = -a, -b, -d
line_rmse = np.sqrt(np.mean((a * line_points[:, 0] + b * line_points[:, 1] -


print('Line [a, b, d]:', np.round([a, b, d], 4))
print('Normal length:', np.sqrt(a*a + b*b))
print('Line inliers:', len(line_points), 'RMSE:', round(line_rmse, 4))
```

```
Line [a, b, d]: [0.7181 0.6959 1.5222]
Normal length: 1.0
Line inliers: 56 RMSE: 0.7255
```

(b) Estimate the circle

Remove the line inliers. Use three-point RANSAC samples to find a circle, then minimize radial error over its consensus.

In [4]:

```
# (b) Remove line inliers, then fit a circle to the remaining points.
remaining = np.flatnonzero(~line_mask)
points = X[remaining]
rng = np.random.default_rng(102)
best_count = 0
best_error = np.inf
for trial in range(2000):
    sample = rng.choice(len(points), 3, replace=False)
    p = points[sample]
    # Subtract circle equations to solve for the centre.
    A = 2 * (p[1:] - p[0])
    B = np.sum(p[1:]**2, axis=1) - np.sum(p[0]**2)
    if np.linalg.cond(A) > 1e8:
        continue  # Skip nearly collinear samples.
    centre = np.linalg.solve(A, B)
    radius = np.linalg.norm(p[0] - centre)
    errors = np.abs(np.linalg.norm(points - centre, axis=1) - radius)
    inliers = errors <= 1.3
    count = inliers.sum()
    error = np.sum(errors[inliers]**2)
    if count >= 25 and (count > best_count or (count == best_count and error
        best_count = count
        best_error = error
        circle_mask = inliers
        circle_sample = sample
        sample_circle = np.array([centre[0], centre[1], radius])
```

```
if best_count == 0:
    raise RuntimeError('No circle with enough inliers was found.')

circle_points = points[circle_mask]
def circle_error(model):
    cx, cy, radius = model
    distances = np.sqrt((circle_points[:, 0] - cx)**2 + (circle_points[:, 1]
    return np.sum((distances - radius)**2)

# Minimize radial error, with radius constrained to be positive.
fit = minimize(circle_error, sample_circle, method='L-BFGS-B',
               bounds=[(None, None), (None, None), (1e-6, None)])
if not fit.success:
    raise RuntimeError(fit.message)
cx, cy, radius = fit.x
circle_rmse = np.sqrt(circle_error(fit.x) / len(circle_points))


print('Circle [cx, cy, radius]:', np.round(fit.x, 4))
print('Circle inliers:', len(circle_points), 'RMSE:', round(circle_rmse, 4))
print('Circle centre error:', round(np.linalg.norm(fit.x[:2] - [2, 3]), 4))
print('Circle radius error:', round(abs(radius - 10), 4))
```

```
Circle [cx, cy, radius]: [1.8683 3.0668 9.9487]
Circle inliers: 44 RMSE: 0.5673
Circle centre error: 0.1477
Circle radius error: 0.0513
```

(c) Show the results together

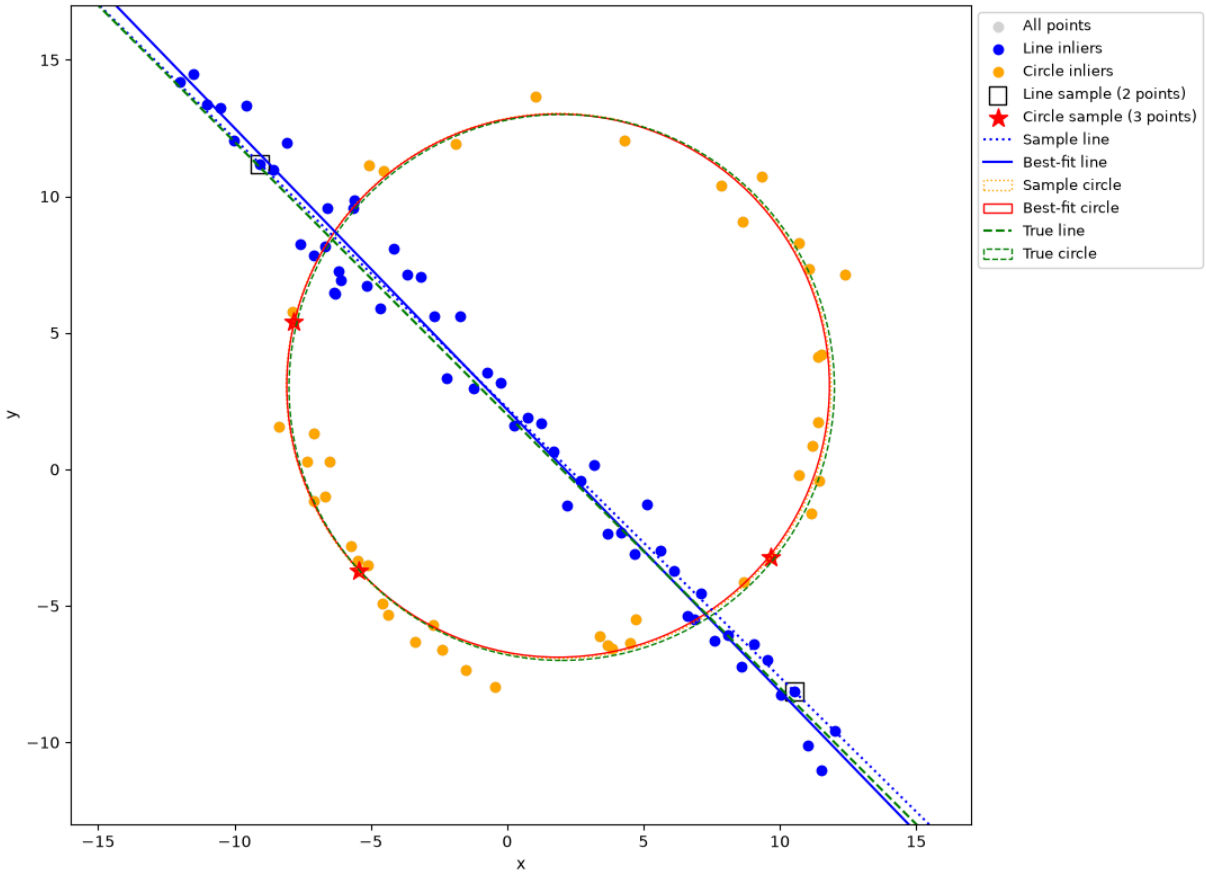
Plot the samples, sample models, refined models and both sets of inliers on the same axes. The true models are included for comparison.

In [5]:

```
# (c) Plot all points, samples, sample models and refined models together.
fig, ax = plt.subplots(figsize=(11, 8))
ax.scatter(X[:, 0], X[:, 1], color='lightgray', label='All points')
ax.scatter(line_points[:, 0], line_points[:, 1], color='blue', label='Line i
ax.scatter(circle_points[:, 0], circle_points[:, 1], color='orange', label='
ax.scatter(X[line_sample, 0], X[line_sample, 1], s=130, marker='s',
           facecolors='none', edgecolors='black', label='Line sample (2 poir
ax.scatter(points[circle_sample, 0], points[circle_sample, 1], s=150,
           marker='*', color='red', label='Circle sample (3 points)')
xx = np.array([-16, 17])
for model, style, label in [(sample_line, 'b:', 'Sample line'),
                            ([a, b, d], 'b-', 'Best-fit line')]:
    aa, bb, dd = model
    if abs(bb) > 1e-10:
        ax.plot(xx, (dd - aa * xx) / bb, style, label=label)
    else:
        ax.axvline(dd / aa, color='blue', label=label)
ax.add_patch(plt.Circle(sample_circle[:2], sample_circle[2], fill=False,
                        color='orange', linestyle=':', label='Sample circle
ax.add_patch(plt.Circle((cx, cy), radius, fill=False, color='red', label='Be
ax.plot(xx, -xx + 2, 'g--', label='True line')
ax.add_patch(plt.Circle((2, 3), 10, fill=False, color='green', linestyle='--
```

```
ax.set_xlim(-16, 17)
ax.set_ylim(-13, 17)
ax.set_aspect('equal')
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.legend(loc='upper left', bbox_to_anchor=(1, 1), fontsize=9)
plt.tight_layout()
plt.show()


# Use the known labels only to evaluate the result.
print('True circle points removed with the line:', np.sum(line_mask[:50]))
print('True line points included in the circle:', np.sum(remaining[circle_ma
```




```
True circle points removed with the line: 6
True line points included in the circle: 0
```

(d) What happens if the circle is fitted first?

Circle-first fitting can work, but nearly collinear points may give a very large circle resembling the line. It can also remove line points near the intersections. Line-first fitting reduces this ambiguity but removes a few true circle points too.

Result: The estimated centre is about (1.868, 3.067) and the radius is 9.949, close to the true (2, 3) and 10. The consensus is kept fixed during refinement. This part asks for an explanation rather than additional code.

Question 3 — Homography and blending

Place the UK flag on Wadham College and a drawn Japanese flag on the Venturi House as fictional celebration/exchange displays. Four wall corners are stored in clockwise order for repeatability; they were manually chosen, not recorded mouse clicks. The commented `ginput` lines allow replacement by four clicks when using an interactive Matplotlib backend. Compare opacity 0.55 and 0.85.

In [6]:
```
%matplotlib inline

import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

wadham = cv.imread('images/wadham.jpg')
venturi = cv.imread('images/venturi.jpg')
venturi = cv.resize(venturi, (1000, round(venturi.shape[0] * 1000 / venturi.
uk = cv.imread('images/flag.png')

# Make a Japanese flag: a red circle on a white background (BGR colours).
japan = np.full((300, 450, 3), 255, dtype=np.uint8)
cv.circle(japan, (225, 150), 90, (45, 0, 188), -1)

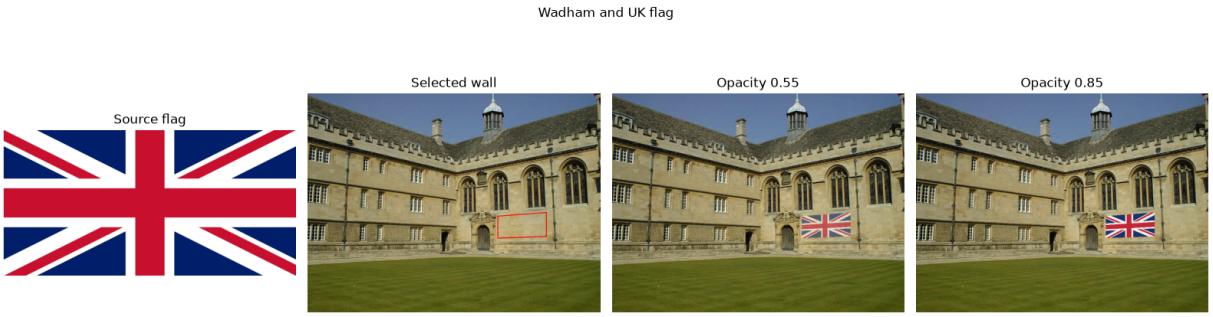
# Wall corners on wadham: top-left, top-right, bottom-right, bottom-left.
wall1 = np.float32([[665, 430], [835, 417], [835, 501], [665, 507]])
# Wall corners on venturi: top-left, top-right, bottom-right, bottom-left.
wall2 = np.float32([[245, 260], [405, 260], [405, 350], [245, 350]])

pairs = [(wadham, uk, wall1, 'Wadham College — UK flag'),
         (venturi, japan, wall2, 'Venturi — Japanese flag')]

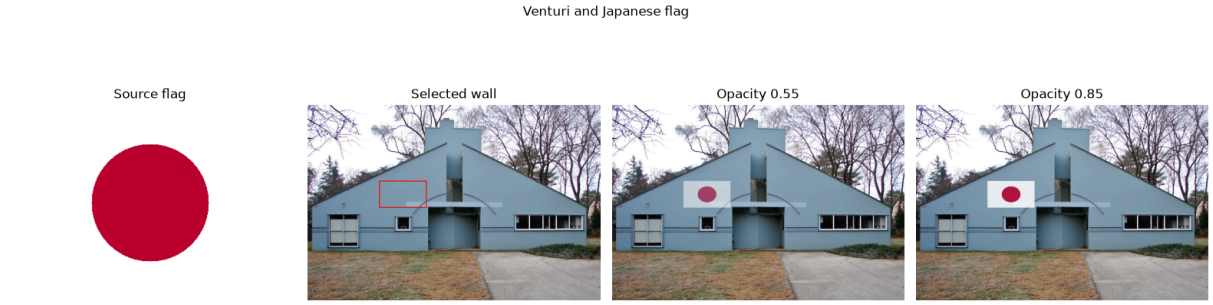
for building, flag, wall, title in pairs:
    height, width = flag.shape[:2]
    corners = np.float32([[0, 0], [width - 1, 0],
                          [width - 1, height - 1], [0, height - 1]])
    H = cv.getPerspectiveTransform(corners, wall)
    bh, bw = building.shape[:2]
    warped_flag = cv.warpPerspective(flag.astype(float), H, (bw, bh))

    # The mask covers the flag area, leaving the rest of the building unchar
    mask = cv.warpPerspective(np.ones((height, width), dtype=float), H, (bw,
    marked = building.copy()
    cv.polylines(marked, [wall.astype(int)], True, (0, 0, 255), 2)
    fig, axes = plt.subplots(1, 4, figsize=(16, 5))
    axes[0].imshow(cv.cvtColor(flag, cv.COLOR_BGR2RGB))
    axes[0].set_title('Source flag')
    axes[1].imshow(cv.cvtColor(marked, cv.COLOR_BGR2RGB))
    axes[1].set_title('Selected wall')
    for ax, opacity in zip(axes[2:], [0.55, 0.85]):
        result = building * (1 - opacity * mask[:, :, None]) + opacity * war
        result = np.clip(result, 0, 255).astype(np.uint8)
        ax.imshow(cv.cvtColor(result, cv.COLOR_BGR2RGB))
```

```
        ax.set_title('Opacity ' + str(opacity))
    for ax in axes:
        ax.axis('off')
    fig.suptitle(title)
    plt.tight_layout()
    plt.show()
    print(title)
    print('Wall corners:', wall)
    print('Homography:\n', np.round(H, 4))
```




```
Wadham and UK flag
Wall corners: [[665. 430.]
 [835. 417.]
 [835. 501.]
 [665. 507.]]
Homography:
 [[ 5.23e-02  0.00e+00  6.65e+02]
 [-2.49e-02  8.03e-02  4.30e+02]
 [-0.00e+00  0.00e+00  1.00e+00]]
```




```
Venturi and Japanese flag
Wall corners: [[245. 260.]
 [405. 260.]
 [405. 350.]
 [245. 350.]]
Homography:
 [[  0.3563   0.     245.    ]
 [  0.       0.301  260.    ]
 [  0.       0.       1.    ]]
```

Discussion: The flag edges follow the selected wall plane. Lower opacity preserves more wall texture, while higher opacity gives stronger flag colours. The transform does not account for folds, shadows or objects in front of the flag, so the result is a simple planar overlay.

Q4)

Use `AECC1.jpg` as the template. Detect SIFT features, match descriptors with `BFMatcher`, keep matches passing the 0.75 ratio test, and estimate a homography with RANSAC (3-pixel threshold). Read images at half resolution to reduce computation. Compare absolute grayscale differences before and after alignment on the same valid overlap.

In [7]:
```
%matplotlib inline

import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

cv.setRNGSeed(230070)
cv.setNumThreads(2)

# Read the template and the five PCB images.
template = cv.imread('images/pcbs/AECC1.jpg', cv.IMREAD_REDUCED_COLOR_2)
files = ['AAAA1.jpg', 'ADAD1.jpg', 'AECC2.jpg', 'AECC3.jpg', 'AECC4.jpg']
images = []
for filename in files:
    image = cv.imread('images/pcbs/' + filename, cv.IMREAD_REDUCED_COLOR_2)
    images.append(image)
```

a)

Read the template and PCB images, convert them to grayscale, and calculate SIFT keypoints and descriptors.

In [8]:
```
# Convert the template and input images to grayscale.
template_gray = cv.cvtColor(template, cv.COLOR_BGR2GRAY)
sift = cv.SIFT_create(nfeatures=7000)
kp1, des1 = sift.detectAndCompute(template_gray, None)

keypoints = []
descriptors = []
for image, filename in zip(images, files):
    gray = cv.cvtColor(image, cv.COLOR_BGR2GRAY)
    kp2, des2 = sift.detectAndCompute(gray, None)
    keypoints.append(kp2)
    descriptors.append(des2)
    print(filename, 'keypoints:', len(kp2))
print('Template keypoints:', len(kp1))
```

```
AAAA1.jpg keypoints: 1428
ADAD1.jpg keypoints: 5490
AECC2.jpg keypoints: 6900
AECC3.jpg keypoints: 6747
AECC4.jpg keypoints: 7000
Template keypoints: 6990
```

b)

Use BFMatcher and the 0.75 ratio test to select matches. Estimate each input-to-template homography with RANSAC and display the accepted matches together.

In [9]:

```
def match_pcb(image, kp2, des2, filename):
    if des2 is None:
        raise RuntimeError('No SIFT descriptors: ' + filename)
    bf = cv.BFMatcher(cv.NORM_L2)
    matches = bf.knnMatch(des2, des1, k=2)
    good = []
    for pair in matches:
        if len(pair) == 2:
            first, second = pair
            if first.distance < 0.75 * second.distance:
                good.append(first)
    if len(good) < 4:
        raise RuntimeError('Not enough matches: ' + filename)

    # Get corresponding coordinates from the accepted descriptor matches.
    source = []
    destination = []
    for match in good:
        source.append(kp2[match.queryIdx].pt)
        destination.append(kp1[match.trainIdx].pt)
    source = np.float32(source).reshape(-1, 1, 2)
    destination = np.float32(destination).reshape(-1, 1, 2)
    cv.setRNGSeed(230070)
    H, inliers = cv.findHomography(source, destination, cv.RANSAC, 3.0,
                                   maxIters=5000, confidence=0.999)
    if H is None or inliers.sum() < 4:
        raise RuntimeError('Homography failed: ' + filename)

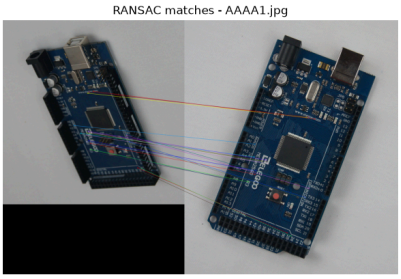
    # Display up to 70 matches to keep the lines readable.
    accepted = []
    for match, keep in zip(good, inliers.ravel()):
        if keep:
            accepted.append(match)
    matched = cv.drawMatches(image, kp2, template, kp1, accepted[:70], None,
                             flags=cv.DrawMatchesFlags_NOT_DRAW_SINGLE_POINT
    print(filename, '- good matches:', len(good), 'RANSAC inliers:', len(acc
    return H, matched

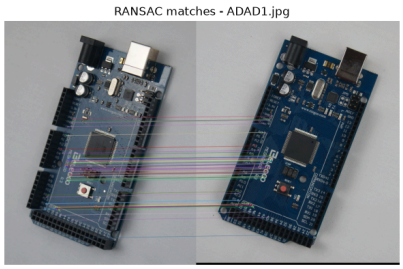

homographies = []
match_images = []
for image, kp2, des2, filename in zip(images, keypoints, descriptors, files)
    H, matched = match_pcb(image, kp2, des2, filename)
    homographies.append(H)
    match_images.append(matched)

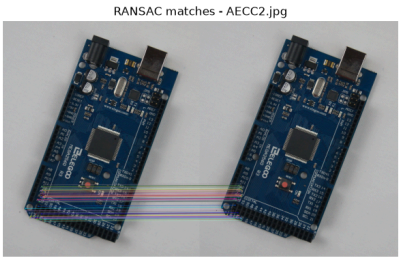
# Show all match results in a single grid, like the reference layout.
fig, axes = plt.subplots(2, 3, figsize=(18, 9))
axes = axes.ravel()
for i, matched in enumerate(match_images):
    axes[i].imshow(cv.cvtColor(matched, cv.COLOR_BGR2RGB))
    axes[i].set_title('RANSAC matches - ' + files[i])
```

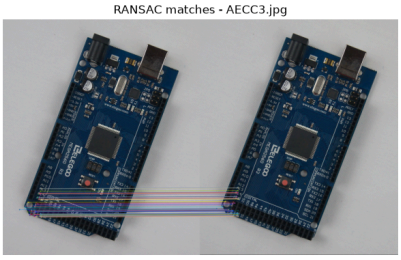
```
    axes[i].axis('off')
axes[-1].axis('off')
plt.tight_layout()
plt.show()
```

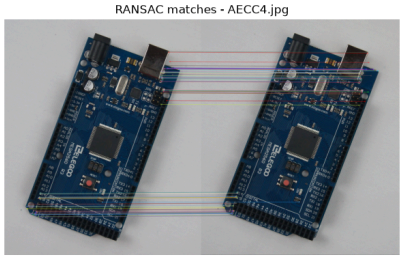
```
AAAA1.jpg - good matches: 54 RANSAC inliers: 23
ADAD1.jpg - good matches: 384 RANSAC inliers: 212
AECC2.jpg - good matches: 3176 RANSAC inliers: 2867
AECC3.jpg - good matches: 3675 RANSAC inliers: 3366
AECC4.jpg - good matches: 3172 RANSAC inliers: 2849
```











c)

Warp each input to the template and use `cv.absdiff` to calculate the grayscale difference. Compare mean differences only where both images have valid pixels. This processes all five images as required in part (d).

In [10]:
```
def show_difference(image, H, filename):
    height, width = template_gray.shape
    gray = cv.cvtColor(image, cv.COLOR_BGR2GRAY)
    aligned = cv.warpPerspective(image, H, (width, height))
    aligned_gray = cv.cvtColor(aligned, cv.COLOR_BGR2GRAY)
    difference = cv.absdiff(aligned_gray, template_gray)

    # Exclude missing regions and interpolation along the image border.
    white = np.ones(gray.shape, dtype=np.uint8) * 255
    valid = cv.warpPerspective(white, H, (width, height), flags=cv.INTER_NEA
    valid = cv.erode(valid, np.ones((5, 5), np.uint8)) > 0
    if not valid.any():
        raise RuntimeError('No valid overlap: ' + filename)
    shown_difference = difference.copy()
    shown_difference[~valid] = 0

    # Compare against a simple resize before alignment.
    resized = cv.resize(gray, (width, height))
    before = cv.absdiff(resized, template_gray)
    print(filename)
```

```
    print('Valid overlap:', round(100 * valid.mean(), 1), '%')
    print('Mean difference before alignment:', round(before[valid].mean(), 2
    print('Mean difference after alignment:', round(difference[valid].mean()

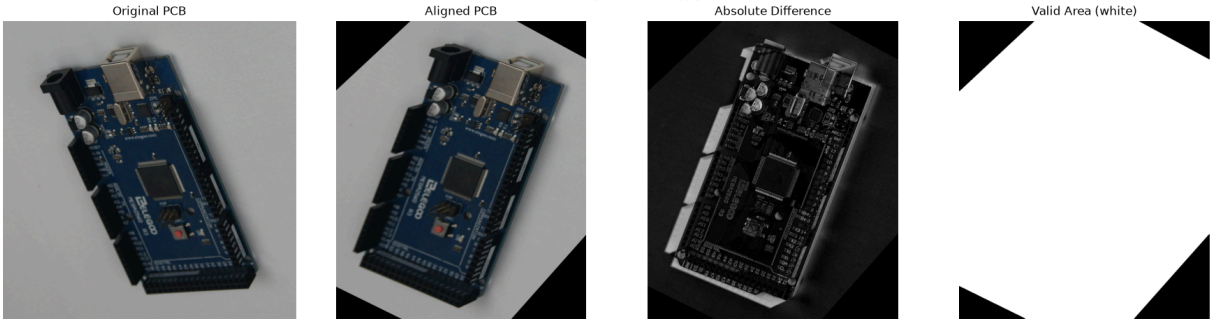
    return aligned, shown_difference, valid


aligned_images = []
difference_images = []
valid_masks = []
for image, H, filename in zip(images, homographies, files):
    aligned, difference, valid = show_difference(image, H, filename)
    aligned_images.append(aligned)
    difference_images.append(difference)
    valid_masks.append(valid)
```

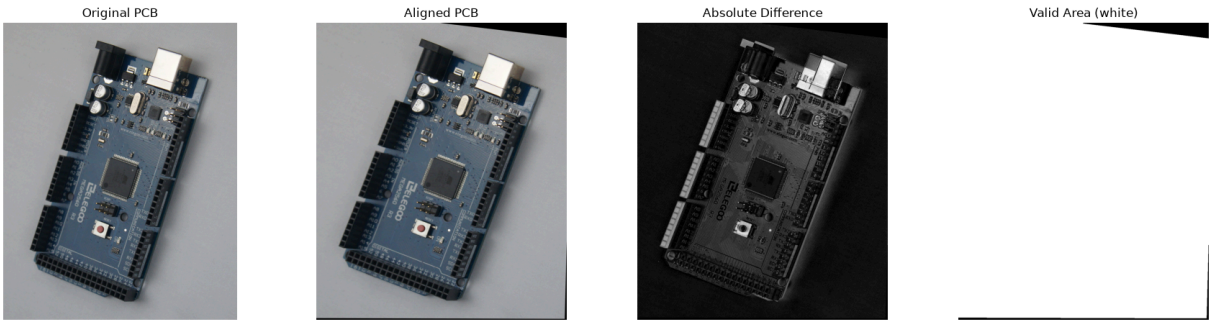
```
AAAA1.jpg
Valid overlap: 88.7 %
Mean difference before alignment: 49.94
Mean difference after alignment: 28.2
ADAD1.jpg
Valid overlap: 97.9 %
Mean difference before alignment: 31.25
Mean difference after alignment: 31.02
AECC2.jpg
Valid overlap: 98.1 %
Mean difference before alignment: 10.01
Mean difference after alignment: 2.53
AECC3.jpg
Valid overlap: 98.6 %
Mean difference before alignment: 8.73
Mean difference after alignment: 2.26
AECC4.jpg
Valid overlap: 98.2 %
Mean difference before alignment: 9.84
Mean difference after alignment: 3.17
```

In [11]:
```
# Display a four-image row for each PCB.
for i, filename in enumerate(files):
    fig, axes = plt.subplots(1, 4, figsize=(18, 5))
    axes[0].imshow(cv.cvtColor(images[i], cv.COLOR_BGR2RGB))
    axes[0].set_title('Original PCB')
    axes[1].imshow(cv.cvtColor(aligned_images[i], cv.COLOR_BGR2RGB))
    axes[1].set_title('Aligned PCB')
    # Auto-scale so even small differences are visible (fixes last 3 PCBs).
    axes[2].imshow(difference_images[i], cmap='gray')
    axes[2].set_title('Absolute Difference')
    # Cast bool mask to uint8 so the valid region shows as white.
    axes[3].imshow(valid_masks[i].astype(np.uint8) * 255, cmap='gray', vmin=
    axes[3].set_title('Valid Area (white)')
    for ax in axes:
        ax.axis('off')
    fig.suptitle('PCB Image - ' + filename, fontsize=15)
    plt.tight_layout()
    plt.show()
```

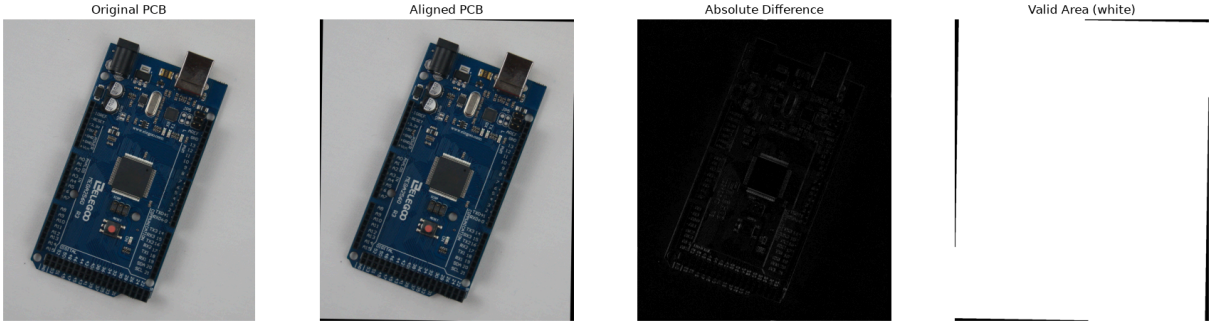
PCB Image - AAAA1.jpg



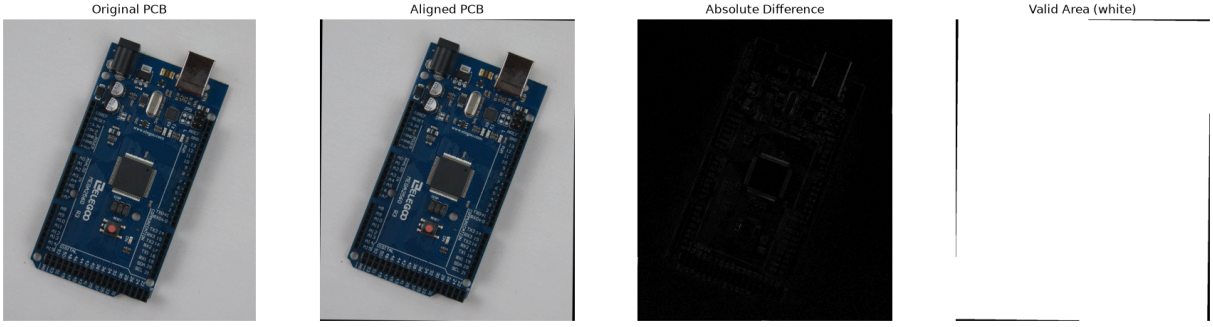
PCB Image - ADAD1.jpg



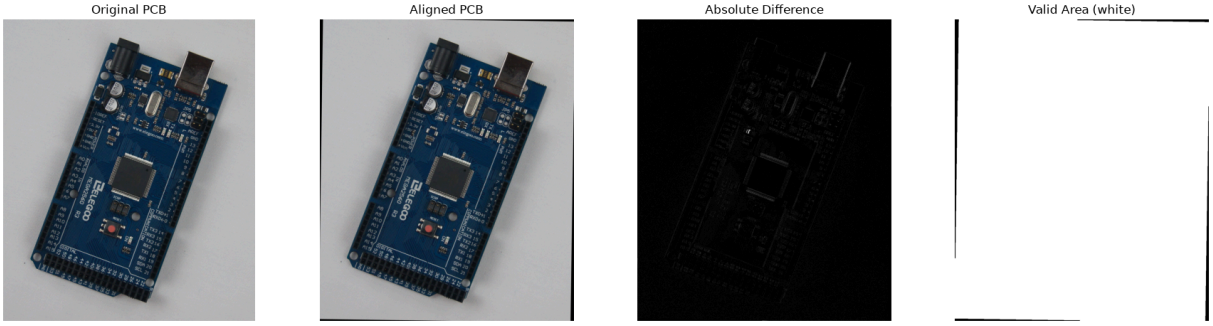
PCB Image - AECC2.jpg



PCB Image - AECC3.jpg



PCB Image - AECC4.jpg



Q4 : Discussion

AECC2–4 align closely, with mean differences of 2.26–3.17. ADAD1 changes only slightly, from 31.25 to 31.02, while AAAA1 improves from 49.94 to 28.20. Viewpoint and appearance differences remain in the latter two images. The invalid area is shown separately and excluded from the averages. A lower mean difference indicates closer pixel agreement, not necessarily fewer defects. The USB port, capacitors and reset button rise above the board plane, so viewpoint changes cause parallax that one homography cannot remove. Reflections, shadows and occlusion also produce large differences. Bright regions therefore need inspection before being labelled as defects.